

LUCRARE DE LABORATOR NR. 2

MODELE DE NEURONI ARTIFICIALI

1. SCOPUL LUCRĂRII

În această lucrare se studiază modele de neuroni artificiali și câteva aplicații ale acestora.

2. BREVIAR TEORETIC

2.1. Modelul de neuron artificial McCulloch-Pitts

Acest model a fost propus de McCulloch și Pitts în 1943 .

Neuronul artificial este modelat ca un dispozitiv neliniar cu mai multe intrări, cu interconexiuni ponderate w_{ij} (fig. 2.1.1):

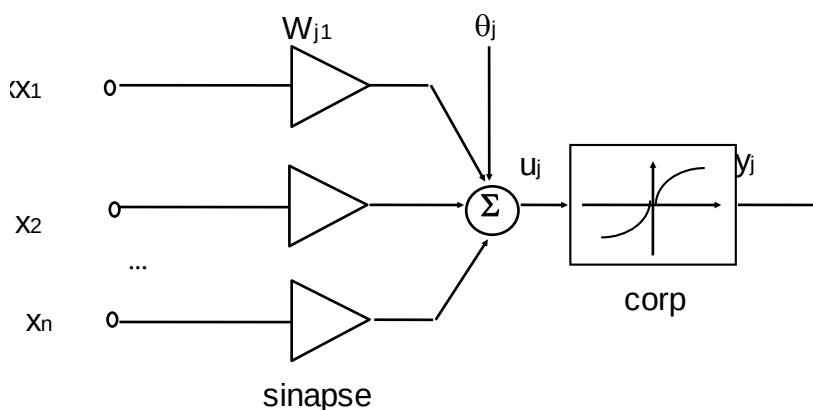


Fig. 2.1.1.

Cel mai simplu model însumează cele n intrări ponderate și transmite rezultatul printr-o neliniaritate la ieșire, conform ecuației:

$$y_j = \psi \left[\sum_{i=1}^n w_{ji} x_i + \theta_j \right]$$

unde ψ este o funcție de limitare sau prag, numită și funcție de activare sau funcție de transfer neliniară; θ_j este un prag extern numit și offset sau bias; w_{ji} sunt ponderile sinaptice ale conexiunilor; x_i sunt intrările în neuron; n este numărul total de intrări și y_j este ieșirea unității de procesare.

Neuronul artificial este caracterizat deci de neliniaritatea sa și

de offsetul θ_j . Modelul McCulloch-Pitts al neuronului folosește ca funcție de activare o funcție prag binară.

În general, funcția prag poate fi înlocuită de o funcție neliniară mai generală și în consecință, ieșirea neuronului poate lua valori într-un set discret $\{-1,1\}$ sau $\{0,1\}$ sau poate varia în mod continuu, de obicei între două valori limita $y_{\min}$ și $y_{\max}$.

Nivelul de activare sau starea neuronului este măsurat prin valoarea semnalului de ieșire y_j . Semnalul de ieșire este în mod uzual determinat aplicând o funcție de tip sigmoidal asupra sumei ponderate a semnalelor de intrare.

Această funcție este descrisă (pentru reprezentarea bipolară) de relația:

$$y_j = \tanh(\alpha u_j) = \frac{1 - e^{-2\alpha u_j}}{1 + e^{-2\alpha u_j}}$$

sau, pentru reprezentarea unipolară, de relația:

$$y_j = \frac{1}{1 + e^{-\alpha u_j}}$$

În cele două relații: $u_j = \sum_{i=0}^n w_{ji} x_i$ iar α este un număr real pozitiv, constant sau variabil, care controlează panta funcției sigmoide.

În comparație cu funcția prag liniară, funcția sigmoidă este mult mai adecvată pentru implementările hard analogice ale neuronilor artificiali; de asemenea este și mai realistă din punct de vedere al plauzibilității biologice. Un astfel de model al neuronului artificial poate fi construit hard folosind componente electronice uzuale.

Mai târziu (după ce în 1943 McCulloch și Pitts și-au formulat modelul lor de neuron artificial) apare noțiunea de **perceptron**, care denumește un strat în care sunt conectați mai mulți neuroni artificiali; fiecare din acești neuroni poate fi numit în acest caz **perceptron simplu**.

2.2. Rețele neuronale MultiLayer Perceptron (MLP)

Rețelele neuronale MLP formează cea mai utilizată clasă de rețele neuronale. Aceste rețele sunt compuse din neuroni artificiali interconectați, organizați pe trei sau mai multe nivele (straturi). Neuronii din primul strat sunt cei care primesc semnale externe.

L2 – Modele de neuroni artificiali

Semnalele externe pot fi semnale de tip analogic (valori continue într-o gamă de intrare) și/sau semnale de tip digital (cu valori binare simbolizate prin “0” sau “1”).

Din punct de vedere al intrărilor, putem clasifica rețelele neuronale MLP în trei clase:

- rețele binare (au doar intrări de tip binar)
- rețele analogice (au doar intrări de tip analogic)
- rețele mixte (au și intrări binare și intrări analogice)

Straturile intermediare de neuroni sunt denumite straturi ascunse, iar ultimul strat neuronal este denumit strat de ieșire.

Neuronii dintr-un strat sunt total conectați cu neuronii din stratul următor. Nu există conexiuni între neuronii din același nivel.

În fig. 2.2.1. este desenată o rețea MLP 2-2-2 (doi neuroni de intrare, un singur strat ascuns cu doi neuroni, doi neuroni în stratul de ieșire).

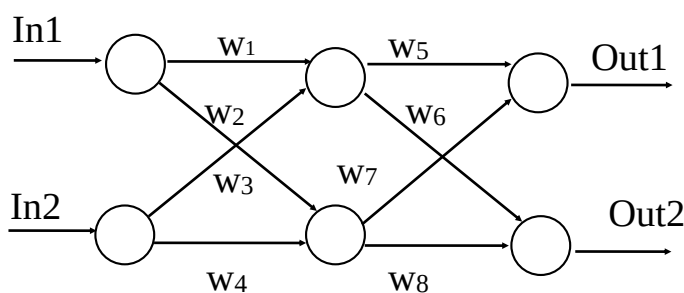


Fig. 2.2.1.

3. DESFĂȘURAREA LUCRĂRII

Se vor edita și apoi executa programele descrise în continuare.

3.1. Programul 1

Scriem un program care, pentru o rețea neuronală ce are un număr oarecare de neuroni în stratul de intrare, un număr oarecare de

L2 – Modele de neuroni artificiali

neuroni în stratul ascuns și un singur neuron în stratul de ieșire, calculează ieșirea rețelei (ilustrează etapa de propagare înainte din algoritmul de backpropagation).

//Reteaua NNI-NNH-1

//Calculul iesirii y

class RN1

{

public static void main(String args[])

{

final int NNI=2;//Numarul de neuroni de intrare

final int NNH=3;//Numarul de neuroni de iesire

//Intrarile, le memoram in vactorul x[]:

double x[]=new double[NNI];

//Atribuim valori oarecare intrarilor (date de test a

//aplicatiei):

x[0]=0.7;

x[1]=0.25;

//Ponderile, primul strat, le memoram in matricea

//w1[][]:

double w1[][]=new double[NNH][NNI];

//Atribuim valori oarecare acestori ponderi (date de

//test):

w1[0][0]=0.5;

w1[0][1]=-0.3;

w1[1][0]=0.4;

```
w1[1][1]=0.3;
```

```
w1[2][0]=0.7;
```

```
w1[2][1]=-0.6;
```

*//Ponderile, al doilea strat, le memoram in
vectorul w2[]. (folosim un vector si nu o matrice,
pentru ca*

//avem un singur neuron de iesire).

```
double w2[]=new double[NNH];
```

//Atribuim valori oarecare acestor ponderi:

```
w2[0]=1.5;
```

```
w2[1]=2.3;
```

```
w2[2]=0.4;
```

//calcul iesiri neuroni din stratul ascuns:

```
double yH[]=new double[NNH];
```

```
for(int i=0;i<NNH;i++){
```

//calcul activare neuron ascuns i:

```
double a=0;
```

```
for(int j=0;j<NNI;j++){
```

```
    a=a+x[j]*w1[i][j];
```

```
    yH[i]=1/(1+Math.exp(-a));
```

```
}
```

//calcul iesire y:

```
double a=0;
```

```
for(int i=0;i<NNH;i++){
```

```
    a=a+yH[i]*w2[i];
```

```
double y=1/(1+Math.exp(-a));  
System.out.println("y="+y);  
}  
}
```

3.2. Programul 2

Scriem un program care, pentru o rețea neuronală ce are un număr oarecare de neuroni în stratul de intrare, un număr oarecare de neuroni în stratul ascuns și un număr oarecare de neuroni în stratul de ieșire, calculează ieșirea rețelei (ilustrează etapa de propagare înainte din algoritmul de backpropagation).

```
//Reteaua NNI-NNH-NNO  
//Calculul iesirilor  
class RN2  
{  
    public static void main(String args[])  
    {  
        final int NNI=2;//nr. neuroni intrare  
        final int NNH=3;//nr. neuroni strat ascuns  
        final int NNO=2;//nr. neuroni iesire  
  
        double x[]=new double[NNI];  
        //initializam intrarile cu numere aleatoare,  
        //in gama (0, 1):
```

```
int i,j;
for(i=0;i<NNI;i++){
    x[i]=Math.random();
    System.out.println("x["+i+"]="+x[i]);
}
//Initializam ponderile cu numere aleatoare
//mici, in gama (-0.5,+0.5):
//Primul strat de ponderi:
double w1[][]=new double[NNH][NNI];
for(i=0;i<NNH;i++){
    for(j=0;j<NNI;j++){
        w1[i][j]=Math.random()-0.5;

System.out.println("w1["+i+"]["+j+"]="+w1[i][j]);
    }
}
//Al doilea strat de ponderi:
double w2[][]=new double[NNO][NNH];
for(i=0;i<NNO;i++){
    for(j=0;j<NNH;j++){
        w2[i][j]=Math.random()-0.5;

System.out.println("w2["+i+"]["+j+"]="+w2[i][j]);
    }
}
//calcul iesirilor neuronilor din stratul ascuns:
double yH[]=new double[NNH];
for(i=0;i<NNH;i++){
```

```
//calcul activare neuron ascuns i:
double a=0;
for(j=0;j<NNI;j++)
    a=a+x[j]*w1[i][j];
yH[i]=1/(1+Math.exp(-a));
}

//calcul iesirilor neuronilor din stratul de
//iesire:
double yO[]=new double[NNO];
for(i=0;i<NNO;i++){
    //calcul activare neuron de iesire i:
    double a=0;
    for(j=0;j<NNH;j++)
        a=a+yH[j]*w2[i][j];
    yO[i]=1/(1+Math.exp(-a));
    System.out.println("yO["+i+"]="+yO[i]);
}
}
```

4.PROBLEME PROPUSE

1. Sa se modifice programele de mai sus astfel incat numarul de neuroni din stratul de intrare, stratul ascuns si stratul de iesire sa se citeasca de la tastatura.
2. Sa se citeasca numarul de neuroni din stratul de intrare, stratul ascuns si stratul de iesire dintr-un fisier text.